Sim2det3D: 3D Traffic Vulnerable Group Detection in Simulation-driven Autonomous Driving

Zexuan Jia (12011126), Bowen Li (12011114), Kaixuan Sun (12012008)

***Abstract*—The perception link is the first critical link in the autonomous driving process. Previous 3D point cloud perception algorithms and data sets tended to improve the detection indicators of the algorithm, while ignoring the perception of small targets such as vulnerable groups in traffic. To solve this problem, the preparation of high-quality datasets of disadvantaged groups becomes key. This article proposes a new joint simulation strategy based on Carla and Apollo to improve the unit data value and model performance. It was verified on the PointPillar algorithm and the Voxel R-CNN algorithm respectively, and achieved the expected results.**

***Index Terms*—Autonomous driving, Simulation, Data Acquisition, 3D Detection**

I. Introduction

Autonomous driving is one of the most exciting topics in the fields of machine learning and deep learning. In recent years, autonomous driving technology has developed rapidly, both in academia and industry. The non-control algorithm links of autonomous driving can be divided into three modules: "perception, prediction, and decision-making" [1]. Due to the data-driven nature of deep learning models, good algorithms need to be supported by high-quality data sets, otherwise the expected results will not be achieved. At present, in the decision-making module, it is recognized that 1 million kilometers of learning can achieve better results [2]. However, in the perceptual module, no such accepted concept exists.

Previous 3D lidar detection algorithms generally ignored vulnerable traffic groups (pedestrians, cyclists). We hope to propose a simple strategy or framework to realize the value of unit data frames in enhanced datasets to improve the algorithm's missed detection of traffic-vulnerable groups.

However, Lidar sensors are expensive equipment and their price was un-affordable for this project team. In addition, Lidar data collection and annotation are extremely difficult. Whether compared with traditional data or autonomous driving image data, it has problems such as being "too sparse, abstract in description, and difficult to visualize" [3]. Especially for traffic-disadvantaged groups, it is extremely difficult to label them from real data.

Therefore, the project uses a simulation environment for data collection and data set preparation [4], and develops a set of simulation environment automatic collection and annotation tools to directly read the 3D position of the target from the memory for annotation. At the same time, based on the low frequency of vulnerable groups in the data set [5], the project built simple scenarios in high dimensions to increase the sample size of vulnerable groups.

In summary, in order to solve the problem of small and unclear samples of traffic vulnerable groups (pedestrians, cyclists) in traditional data sets. This project built a set of self-made collection tools based on the simulation environment for data collection, and proposed a set of strategic frameworks. In order to verify the effect, the self-made data set was verified on multiple algorithms, and it was observed that the detection effect of traffic vulnerable groups was significantly improved.

Zexuan Jia, Kaixuan Sun are with the Department of Computer Science and Technology, and Bowen Li is with Electronics and Electrical Engineering, Southern University of Science and Technology, Shenzhen, 518055, China

II. Related Work

Since Carla [6] is a powerful open-source simulator for autonomous driving research, which can create a virtual urban environment and simulate sensors such as cameras, Lidar, and mmWave radar to provide information. Many researchers have designed their self-driving system in the environment of Carla. By simply using mature object detection algorithms such as YOLO [7] and Faster R-CNN [8] to process the data provided by Carla, they can achieve the object detection functionality in self-driving system. This is also the case for object tracking, they can use algorithms such as GOTURN and Deep SORT to achieve the goal.

In this project, the open source 3D point cloud detection algorithm training framework OpenPCDet [9] is used. This framework is currently a popular lightweight point cloud algorithm framework and supports a variety of network training.

Model	Car@R11	Pedestrian@R11	Cyclist@R11	Dataset
PointPillar [10]	77.28	52.29	62.68	KITTI
SECOND [11]	78.62	52.98	67.15	KITTI
Voxel R-CNN [12]	84.54	-	-	KITTI
BEVFusion [13]	67.75	-	-	nuScenes
CenterPoint [14]	78.08	49.74	67.22	ONCE
Voxel NeXt [15]	30.05	-	-	Argoverse2

TABLE I: 3D Lidar detection models OpenPCDet provided

OpenPCDet provides some popular pre-trained models (Table: I). It is obviously found that although many models have reached a very high level in terms of indicators, they ignore the detection of vulnerable traffic groups (pedestrians, cyclists). For autonomous driving applications, in the perception module, the missed detection of vulnerable groups is often much more serious than the missed detection of challenging vehicles. It needs more attention.

Among these, we chose the PointPillar model for experiments to verify that our method has improved the detection effect of if groups. Voxel R-CNN was selected for experiments to verify that the traditional vehicle detection effect has also been improved.

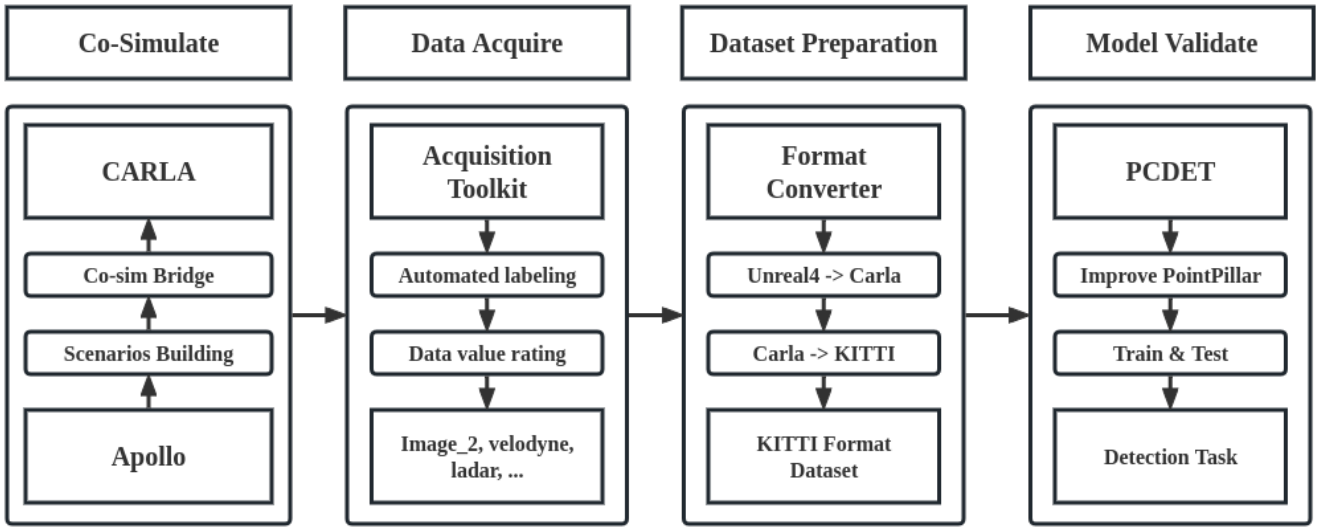


Fig. 1: Main Framework of the Project

III. Methodology

A. Main Framework

1) Co-Simulate: This part mainly builds the simulation scene. Since there are many static vehicles in the Carla map, they are part of the map and are not generated under program control. When annotating, its 3D position does not appear in the memory and cannot be annotated. During the training and verification process, there may be situations where the model detects static vehicles but misjudges them due to missing annotations.

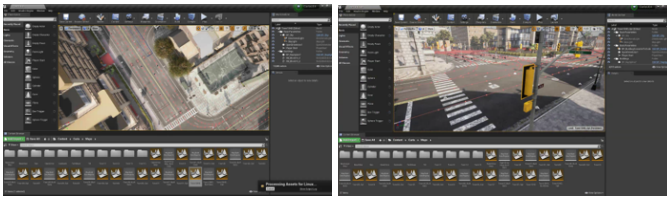


Fig. 2: Operate source code version Carla in the Unreal4

Therefore, we first operate in the source version of Carla and use the Unreal4 [16] toolkit to eliminate static vehicles (Fig: 2).

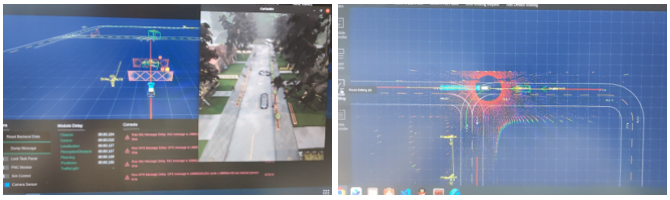


Fig. 3: Apollo-Carla Co-simulation Using Bridge

After that, we used the Carla-Apollo Bridge to let Apollo [17] take over Carla's dynamic scene settings and perform visual operations in DreamViewer 7c.

2) Data Acquire: This part mainly carries out data collection work. This part mainly carries out data collection work. Based on the characteristics of Carla, we created a set of collection tools that can collect online and automatically label. We also used some proactive strategies to optimize the data set to a certain extent [18].

The collected images and Lidar point cloud data form a multi-modal data set, although we did not use image data in the end.

3) Dataset Preparation: This part mainly prepares data sets that comply with standard format (KITTI). This part mainly prepares data sets that comply with standard formats. Carla is a tool developed based on the UE4 engine, and the coordinates of its internal Actors are in the UE4 coordinate system. For the Lidar sensor in Carla, there is no directional information inside UE4, but it exists as a point.

Therefore, in the first step, the annotation coordinates need to be multiplied by the complex deflection angle rotation matrix of the Lidar sensor; in the second step, the annotation position is converted to another right-handed system under the KITTI coordinate system; in the third step, the annotation format is converted to the KITTI format .

4) Model Validate: This part mainly conducts model training and evaluation. This part mainly conducts model training and evaluation. Under the OpenPCDet framework, the PointPillar and Voxel R-CNN algorithms are trained and tested.

B. Problem Definition

Given a 3D point cloud, $\mathbf{P} = \{p_1, p_2, ..., p_n\}$, which represents the set of measured points, $p_i = (x_i, y_i, z_i)$, and presents a snapshots of the surroundings. For objects, $\mathbf{O} = \{o_1, o_2, ..., o_m\}$, represents the set of all objects in point cloud (vehicle, traffic light, pedestrian, etc.), using KITTI format, where $o_j = (x_j, y_j, z_j, w_j, h_j, l_j, r_j)$.

Use S_{keep} to measure whether to keep the Lidar data.

$$S_{keep} \equiv \mathcal{E}(P) \cdot \mathcal{P}(\rho(O), \tau(O))$$

where $\mathcal{E}$ decides whether to keep the entire point cloud, and $\mathcal{P}$ decides whether to keep the collected target data.

Use S_{value} to represent the value of current Lidar data for the trainning model,

$$S_{value} \equiv \phi(P, O) \cdot \psi(O)$$

where $\phi(P, O)$ is the perception distance term, used to describe the relationship between the effective perception radius and the farthest perception radius; $\psi(O)$ is the prediction accuracy term, used to describe the perception How accurate is the prediction of objects within the radius.

C. Pedestrian Walking Control Algorithm

For pedestrian control in the Co-Simulate link, there is the following algorithm (Alg: 1). It controls pedestrian behavior during the time between two timestamps, where pos is the position, $range$ is perception range (only 160 degrees in front of the eyes), $towards$ is the absolute angle of orientation, and $speed$ is the speed of a pedestrian, respectively.

Algorithm 1: Simple Pedestrian Control

```
Result: towards, speed
Initialize: pos, range, towards, speed
speed ← norm speed
 acss ← Set(distance(ac_i, pos) < range.dist)
for ac_i in acss do
    if ac_i is trafficlight then
        towards ← Rad(radius, ac_i)
          speed ← corss speed return
    end
    else
        if ac_i is corner then
            towards ← Rand(straight, turn)
              speed ← speed
        end
    end
end
```

D. PointPillar VFE Encoder Upgrade

The following is the entire process of our improved VFE processing stage.

1) De-mean encoding $(P, M, 4) \rightarrow (P, M, 3)$ for each point in Pillar
2) Decentralize the effective points in each Pillar $(P, M, 4) \rightarrow (P, M, 2)$
3) Mask merge coding: Combine the original $(P, M, 4)$ with the above two codes cat to get the vector of $(P, M, 9)$. There are two points to note here:
   a) Only valid points (n points per pillar) are operated in each Pillar. If the number of valid points is insufficient, zero will be added, if there are too many, random sample will be used;
   b) In the code, **the 9-dimensional encoding vector is The first 2 dimensions are replaced by decentralized encoded vectors** (Our work)
4) Convolution kernel pooling: $(P, M, 9) \rightarrow (P, M, 64)$ and $(P, 64)$
5) Pillarscatter: Go to the 2D feature map of $(X/vsize, Y/vsize)$ and get the feature map of $(64, X/vsize, Y/vsize)$.

The principle is that the PointPillar model uses Pillar that is different from Voxel to describe the point cloud, ignoring some information on the Z axis. When performing VFE encoding, because the model does not use this information, the encoding can also ignore this part. This achieves the effect of improving coding speed and reducing training and inference time.

E. Post-processing Uncertainty Noise Addition

In order to make the data set more comprehensive and cover the environment that affects the Lidar sensor under various weather conditions, the project performs uncertainty post-processing on some data under extreme weather conditions [19].

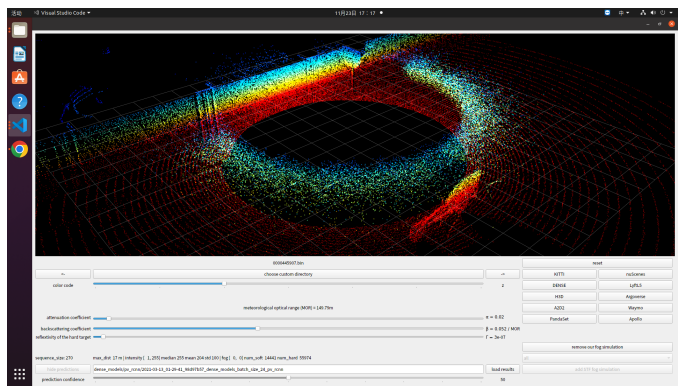


Fig. 4: Main Framework of the Project

According to the DENSE data set and the previously provided example, it is abstracted into two parameters α and β.

IV. Experimental Setup

A. Dataset Preparation

A total of three data sets were collected, of which A did not use the scenes we built, and B and D used the scenes we built. Each data set consists of 5 subsets, and the number of "vehicles (including cyclists) and pedestrians" in each subset are $(50, 25)$, $(75, 37)$, $(100, 50)$, $(125, 62)$ and $(150, 75)$.

Dataset name	Total Frames	Map	Detail
A	987	Town05	City
B	902	Town02	Town
D	988	Town06	Highway
V	375	-	Random select from A,B, D

TABLE II: Dataset Configuration and Details

The detailed settings are shown in the table (Tab: II) above.

B. Experiment Setup

Use dataset A,B and D train on PointPillar and Voxel R-CNN respectively. Using $epoch = 160$, $batch\ size = 18$, dynamically adjust learning rate, $Random\ seed = 114$. Totally 6 models.

C. Experiment Platform

All model training is performed on the server, and the server parameters are as follows.

- CPU: Intel(R) Xeon(R) CPU E5-2690 v4 @ 2.60GHz
- GPU: NVIDIA TITAN V $\times$ 6
- OS: Ubuntu 22.04.2 LTS
- MEM: 453 G

V. Results

A. Training Loss Curve

Using the TensorBoard tool, the following is the exported curve of the loss decreasing as the step increases.

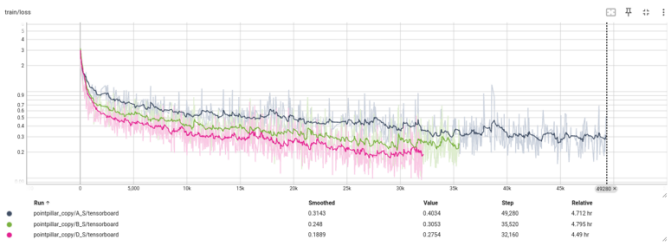


Fig. 5: Training loss curve of PointPillar

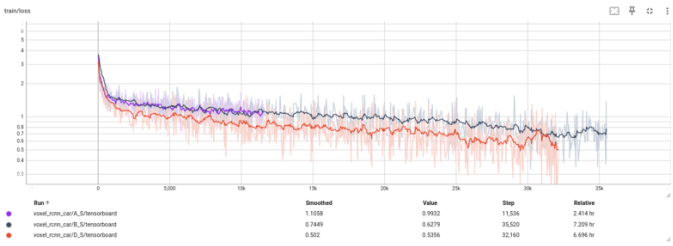


Fig. 6: Training loss curve of Voxel R-CNN

B. Evaluating Result

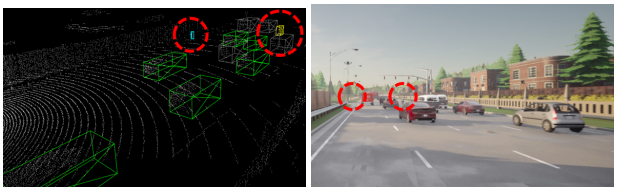
The following are the evaluation results after training in OpenPCDet. The indicator uses mAP70, that is, mAP above 70 is calculated as correct recognition.

Dataset	Car	Truck	Van	Pedestrian	Cyclist
A	53.86	68.32	52.78	38.08	45.11
B	60.28	71.27	54.11	40.87	**56.30**
D	64.09	68.59	71.09	**48.05**	52.26

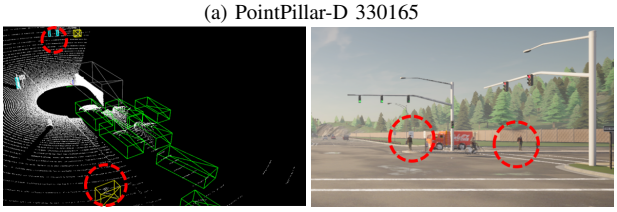
TABLE III: PointPillar evaluating result

Dataset	Vehicle
A	8.63
B	**64.28**
D	63.21

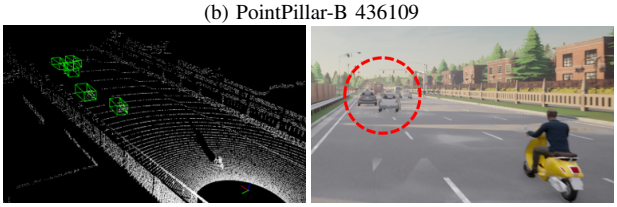
TABLE IV: Voxel R-CNN evaluating result



(a) PointPillar-D 330165



(b) PointPillar-B 436109



(c) Voxel R-CNN-D 003861

Fig. 7: Visualization of different models trained on different data sets under the numbered data frames shown

C. Visualization

Visual display of some data of the model on the test set (Fig: 7).

It is not difficult to find that the PointPillar model can basically accurately detect small objects in the distance. Voxel R-CNN has correctly and completely detected all vehicles in this scene, even those with severe occlusion.

VI. Discussion

A. Method: Pedestrian Agent Control

In this project, in order to flexibly control the behavior of the pedestrian agent, a lot of effort and time was spent, which resulted in the writing of the agent control script being hasty. A pedestrian agent whose behavior has certain uncertainties can be trained through reinforcement learning methods to achieve an upgrade in behavioral control.

B. Result: Voxel R-CNN worse than PointPillar

In the experimental session, we used the A, B, and D data sets to train the PointPillar algorithm and the Voxel R-CNN algorithm respectively. However, the performance of the Voxel R-CNN algorithm that appeared later is actually worse than PointPillar, especially because Voxel R-CNN is still a two-stage algorithm. Observing the Loss curve, you can know that the reason is that the loss value does not decrease as much as

PointPillar. It can be considered that the loss function of Voxel R-CNN is quite different from the loss function of PointPillar. For the same data, the degree of convergence on PointPillar is greater than that of Voxel R-CNN. To further improve the model effect, more data is needed for Voxel R-CNN training.

C. Result: The effect worse than OpenPCDet

Obviously, our single data set only has 5G, which is far smaller than the current open source data set. Naturally, the effect of the trained model is worse than the model provided by OpenPCDet that has been trained on the open source data set for a long time. However, it turns out that our model does not perform significantly worse than the provided model, probably because:

- The data generation in the simulation environment is too standardized and lacks the influence of many uncertain factors in the display environment.
- Our method is more effective, improves the training value of unit data frame, and makes up for some shortcomings.

In addition, it can be seen that the effect of model A of Voxel R-CNN is very poor. This is due to data loss due to power outage in the server room and the termination of model training.

VII. Conclusions

In this paper, in view of the current general neglect of vulnerable traffic groups (pedestrians, cyclists) in 3D detection algorithms, we propose a project framework based on the Carla simulation environment, from scene construction to data preparation and finally to model training. Experiments were conducted on the PointPillar algorithm and Voxel R-CNN algorithm under the OpenPCDet framework. Experimental results show that our algorithm is more effective, and the 3D detection effect for traffic vulnerable groups is significantly improved; at the same time, the effect is also improved for general vehicle detection tasks.

In the future, we will extend this work with other AI technologies, such as knowledge graph [20], [21] and human-centered AI [5] to reduce the reality gap [22], [4].

References

[1] G. Lan, J. Benito-Picazo, D. M. Roijers, E. Domínguez, and A. Eiben, "Real-time robot vision on low-performance computing hardware," in *2018 15th international conference on control, automation, robotics and vision (ICARCV)*. IEEE, 2018, pp. 1959–1965.

[2] G. Lan, J. Sun, C. Li, Z. Ou, Z. Luo, J. Liang, and Q. Hao, "Development of uav based virtual reality systems," in *2016 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. IEEE, 2016, pp. 481–486.

[3] G. Lan, Y. Bu, J. Liang, and Q. Hao, "Action synchronization between human and uav robotic arms for remote operation," in *2016 IEEE International Conference on Mechatronics and Automation*. IEEE, 2016, pp. 2477–2481.

[4] G. Lan, J. Chen, and A. E. Eiben, "Simulated and real-world evolution of predator robots," in *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2019, pp. 1974–1981.

[5] G. Lan, Y. Wu, F. Hu, and Q. Hao, "Vision-based human pose estimation via deep learning: A survey," *IEEE Transactions on Human-Machine Systems*, 2022.

[6] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., vol. 78. PMLR, 13–15 Nov 2017, pp. 1–16.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.

[8] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1440–1448.

[9] O. D. Team, "Openpcdet: An open-source toolbox for 3d object detection from point clouds," https://github.com/open-mmlab/OpenPCDet, 2020.

[10] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "Pointpillars: Fast encoders for object detection from point clouds," *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12 689–12 697, 2018.

[11] Y. Yan, Y. Mao, and B. Li, "Second: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, 2018.

[12] J. Deng, S. Shi, P.-C. Li, W. gang Zhou, Y. Zhang, and H. Li, "Voxel r-cnn: Towards high performance voxel-based 3d object detection," *ArXiv*, vol. abs/2012.15712, 2020.

[13] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. Rus, and S. Han, "Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2774–2781, 2022.

[14] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3d object detection and tracking," *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11 779–11 788, 2020.

[15] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia, "Voxelnext: Fully sparse voxelnet for 3d object detection and tracking," *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 21 674–21 683, 2023.

[16] P. Gestwicki, "Unreal engine 4 for computer scientists," *J. Comput. Sci. Coll.*, vol. 35, no. 5, p. 109–110, oct 2019.

[17] H. Fan, F. Zhu, C. Liu, L. Zhang, L. Zhuang, D. Li, W. Zhu, J. Hu, H. Li, and Q. Kong, "Baidu apollo em motion planner," *ArXiv*, vol. abs/1807.08048, 2018.

[18] J. Lai, Z. Jia, and B. Li, "Active data acquisition in autonomous driving simulation," *ArXiv*, vol. abs/2306.13923, 2023.

[19] M. Hahner, C. Sakaridis, D. Dai, and L. V. Gool, "Fog simulation on real lidar point clouds for 3d object detection in adverse weather," *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 15 263–15 272, 2021.

[20] T. Liu, G. Lan, K. A. Feenstra, Z. Huang, and J. Heringa, "Towards a knowledge graph for pre-/probiotics and microbiota–gut–brain axis diseases," *Scientific Reports*, vol. 12, no. 1, p. 18977, 2022.

[21] G. Lan, T. Liu, X. Wang, X. Pan, and Z. Huang, "A semantic web technology index," *Scientific reports*, vol. 12, no. 1, p. 3672, 2022.

[22] G. Lan, J. Chen, and A. Eiben, "Evolutionary predator-prey robot systems: From simulation to real world," in *Proceedings of the genetic and evolutionary computation conference companion*, 2019, pp. 123–124.